



INFORMS Journal on Computing

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Incremental Upper Bound for the Maximum Clique Problem

Chu-Min Li, Zhiwen Fang, Hua Jiang, Ke Xu

To cite this article:

Chu-Min Li, Zhiwen Fang, Hua Jiang, Ke Xu (2018) Incremental Upper Bound for the Maximum Clique Problem. INFORMS Journal on Computing 30(1):137-153. <https://doi.org/10.1287/ijoc.2017.0770>

Full terms and conditions of use: <http://pubsonline.informs.org/page/terms-and-conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2017, INFORMS

Please scroll down for article—it is on subsequent pages



INFORMS is the largest professional society in the world for professionals in the fields of operations research, management science, and analytics.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Incremental Upper Bound for the Maximum Clique Problem

Chu-Min Li,^{a,b} Zhiwen Fang,^{b,c} Hua Jiang,^a Ke Xu^c

^aSchool of Computer Science, Huazhong University of Science and Technology (HUST), Wuhan 430073, China; ^bModélisation, Information and Systèmes (MIS), EA 4290, Université de Picardie Jules Verne, Amiens 80000, France; ^cState Key Laboratory of Software Development Environment, Beihang University, Beijing 100191, China

Contact: chu-min.li@u-picardie.fr,  <http://orcid.org/0000-0002-6886-8434> (C-ML); zhiwenf@gmail.com (ZF); jh_hgt@163.com (HJ); kexu@nlsde.buaa.edu.cn (KX)

Received: November 23, 2015

Revised: August 14, 2016; April 2, 2017

Accepted: June 12, 2017

Published Online: December 20, 2017

<https://doi.org/10.1287/ijoc.2017.0770>

Copyright: © 2017 INFORMS

Abstract. The maximum clique problem (MaxClique for short) consists of searching for a maximum complete subgraph in a graph. A branch-and-bound (BnB) MaxClique algorithm computes an upper bound of the number of vertices of a maximum clique at every search tree node, to prune the subtree rooted at the node. Existing upper bounds are usually computed from scratch at every search tree node. In this paper, we define an incremental upper bound, called IncUB, which is derived efficiently from previous searches instead of from scratch. Then, we describe a new BnB MaxClique algorithm, called IncMC2, which uses graph coloring and MaxSAT reasoning to filter out the vertices that do not need to be branched on, and uses IncUB to prune the remaining branches. Our experimental results show that IncMC2 is significantly faster than algorithms such as BBMC and IncMaxCLQ. Finally, we carry out experiments to provide evidence that the performance of IncMC2 is due to IncUB.

History: Accepted by David Alderson, Area Editor for Network Optimization: Algorithms and Applications.

Funding: The work was supported by Chinese State Key Laboratory of Software Development Environment Open Fund [Grant SKLSDE-2012KF-07], the National Natural Science Foundation of China [Grants 61370183, 61370184, and 61472147], and by the MeCS platform of the Université de Picardie Jules Verne.

Supplemental Material: The online supplement is available at <https://doi.org/10.1287/ijoc.2017.0770>.

Keywords: MaxClique • MaxSAT • incremental upper bound

1. Introduction

Consider an undirected graph $G = (V, E)$, where V is a set of vertices $\{v_1, v_2, \dots, v_{|V|}\}$ and E is a set of edges. A clique of G is a subset C of V in which every two vertices are adjacent. The size of a clique C is the number of vertices in C , usually denoted by $|C|$. The maximum clique problem (MaxClique for short) consists in finding a clique of G of the largest size. The number of vertices in a maximum clique of G is usually denoted by $\omega(G)$ and is called the *clique number* of G .

MaxClique is a very important NP-hard combinatorial problem, because it appears in many real-world applications such as bioinformatics (e.g., Strickland et al. 2005) and fault diagnosis (e.g., Berman and Pelc 1990). A huge amount of effort has been devoted to solve it. Two main types of algorithms (also called solvers) are developed for MaxClique: heuristic methods (e.g., Pullan and Hoos 2006; Grosso et al. 2008; Pullan et al. 2011; Benlic and Hao 2013), and exact algorithms including Branch-and-Bound (BnB) algorithms (e.g., San Segundo et al. 2011; Tomita and Seki 2003; Tomita and Kameda 2007; Tomita et al. 2010; Li and Quan 2010a; Li and Quan 2010b; Konc and Janezic 2007; Östergård 2002; Carraghan and Pardalos 1990;

Fahle 2002; Régim 2003; McCreesh and Prosser 2014. A survey of algorithms for MaxClique can be found in Wu and Hao (2015). Techniques in constraint programming can also be used in MaxClique solving (see e.g., Régim 2003; McCreesh and Prosser 2014. In this paper, we focus on exact MaxClique algorithms based on the BnB scheme, which guarantees the optimality of the solutions found. See Prosser (2012) for a computational study on exact MaxClique algorithms.

During the search, a BnB MaxClique algorithm computes an upper bound of the clique number of a subgraph G' of G at each search tree node, in order to prune the subtree rooted at the node. The most popular upper bound is obtained by partitioning G' into independent sets and is denoted by UB_{IS} in this paper, where an independent set is a subset of vertices in which no vertex is adjacent to another. If G' can be partitioned into r independent sets, then $UB_{IS} = r$, because any clique in G' contains at most one vertex from each independent set.

Given a partition of G' into UB_{IS} independent sets, a maximum clique of G' often cannot contain a vertex from every independent set, as illustrated in Li and Quan (2010a, b). Let q and h be positive integers.

A subset of q independent sets that cannot form a clique of q vertices is said *conflicting*. MaxSAT reasoning is then proposed in Li and Quan (2010a, b) to detect disjoint conflicting subsets of independent sets. If h disjoint conflicting subsets of independent sets are found, the obtained upper bound is $UB_{IS} - h$ and is denoted by UB_{MaxSAT} in this paper.

The upper bound UB_{IS} is computed from scratch at every search tree node, and the upper bound UB_{MaxSAT} is computed from UB_{IS} . In Li et al. (2013), we proposed an incremental upper bound, denoted by IncUB, that is not computed from scratch, but is derived from the results of previous searches in G . Then, we combined IncUB with UB_{IS} and UB_{MaxSAT} in a portfolio to prune searches in a BnB MaxClique algorithm called IncMaxCLQ. At every search tree node, IncMaxCLQ computes $UB = \min(\text{IncUB}, UB_{IS}, UB_{MaxSAT})$ for every vertex v of the current subgraph. If UB cannot promise a solution better than the largest clique found so far in G , then branching on v is not necessary.

In this paper, we extend Li et al. (2013) and propose a new BnB MaxClique algorithm called IncMC2 that exploits IncUB, UB_{IS} , and UB_{MaxSAT} differently from IncMaxCLQ. At every search tree node, IncMC2 determines a lower bound r of the clique number of the current subgraph G' , and uses independent set partition and MaxSAT reasoning to partition the vertices of G' into two sets A and B , in such a way that A cannot form any clique of size larger than r , and that IncMC2 needs only to branch on the vertices in B . Then, before branching on a vertex in B , IncMC2 computes IncUB for the vertex and uses the computed IncUB to prune the branch.

The difference between IncMaxCLQ and IncMC2 can roughly be summarized as follows. While IncMaxCLQ computes $\min(\text{IncUB}, UB_{IS}, UB_{MaxSAT})$ for every vertex v in G' to prune the branch on v , IncMC2 uses independent set partition and MaxSAT reasoning to filter out the vertices that do not need to be branched on, and uses IncUB to prune the branches on the vertices in B .

We organize this paper as follows to present IncMC2 and its new approach to exploit IncUB for solving MaxClique. In Section 2, we introduce some necessary notations and preliminaries. In Section 3, we describe a basic BnB scheme for MaxClique. In Section 4, we review related work on upper bounds and vertex orderings for MaxClique. In Section 5, we define IncUB and explain how to make IncUB tighter. In Section 6, we describe IncMC2. In Section 7, we discuss the vertex orderings for an effective exploitation of IncUB in IncMC2. In Section 8, we present experimental results to compare IncMC2 with IncMaxCLQ and other BnB MaxClique algorithms and to analyze the effect of IncUB in IncMC2. Finally, we conclude the paper in Section 9.

2. Preliminaries

Let $G = (V, E)$ be a graph and P be a subset of V , the subgraph of G induced by P is defined as $G[P] = (P, E')$, where $E' = \{\{v_i, v_j\} \in E \mid v_i, v_j \in P\}$. The set of adjacent (or neighbor) vertices of a vertex v in G is denoted by $\Gamma(v) = \{u \mid \{u, v\} \in E\}$. The cardinality $|\Gamma(v)|$ of $\Gamma(v)$ is called the degree of v . The density of G is $2 \times |E| / (|V| \times (|V| - 1))$. An independent set (IS) of G is a subset of V in which no two vertices are adjacent. A clique of G is an IS of its complement graph $\bar{G}$, and vice versa, where $\bar{G} = (V, \bar{E})$ with $\bar{E} = \{\{v_i, v_j\} \in V \times V \mid i \neq j \text{ and } \{v_i, v_j\} \notin E\}$.

Given G , the graph coloring problem (GCP) asks to find the minimum number of colors (i.e., the chromatic number $\chi(G)$) necessary to color the vertices of G such that no two adjacent vertices share the same color. Solving GCP is equivalent to partitioning G into a minimum number of ISs. In fact, all vertices sharing the same color in G constitute an IS which is also called a color class. Since a clique can take at most one vertex from each IS, the following holds.

Proposition 1. In any graph G , $\omega(G) \leq \chi(G)$.

3. A Basic Branch-and-Bound Scheme for MaxClique

Given a graph $G = (V, E)$, the objective of a BnB MaxClique algorithm is to find a maximum clique in G by efficiently traversing the search space formed by all the proper subsets of V . For this purpose, the algorithm works with a vertex ordering O over $V = \{v_1, v_2, \dots, v_{|V|}\}$ such that $v_1 < v_2 < \dots < v_{|V|}$. For $i = |V|, |V| - 1, \dots, 1$, it branches on v_i to recursively search for a maximum clique containing v_i in $G[\{v_i, v_{i+1}, \dots, v_{|V|}\}]$. The largest clique among these $|V|$ cliques is then a maximum clique of G . Note that a maximum clique containing v_i in $G[\{v_i, v_{i+1}, \dots, v_{|V|}\}]$ is obtained by adding a maximum clique of $G[\Gamma(v_i) \cap \{v_{i+1}, \dots, v_{|V|}\}]$ into $\{v_i\}$, where $C = \{v_i\}$ is called a growing clique and $P = \Gamma(v_i) \cap \{v_{i+1}, \dots, v_{|V|}\}$ is called a candidate set, because the vertices in P are candidate to be added in C to form a clique of G .

In the general case, let $C = \{c_1, c_2, \dots, c_{|C|}\}$ be a growing clique and $P = \{p_1, p_2, \dots, p_{|P|}\}$ be a candidate set satisfying $P \subseteq \Gamma(c_1) \cap \Gamma(c_2) \cap \dots \cap \Gamma(c_{|C|})$ and $p_1 < p_2 < \dots < p_{|P|}$ w.r.t. O . The algorithm successively branches on p_i (for $i = |P|, |P| - 1, \dots, 1$) to find a maximum clique in $G[\Gamma(p_i) \cap \{p_{i+1}, \dots, p_{|P|}\}]$, which is to be added in $\{p_i\} \cup C$ to form a clique of G .

Let $C_{\max}$ be the largest clique found so far in G . To speed up the search, the algorithm computes an upper bound UB of the size of a maximum clique in $G[\Gamma(p_i) \cap \{p_{i+1}, \dots, p_{|P|}\}]$ when branching on p_i . If $UB < |C_{\max}| - |C|$, a clique larger than $C_{\max}$ obviously cannot be found by branching on p_i . The branch is then pruned.

Algorithm 1 shows the pseudocode of such a procedure. When implementing Algorithm 1, we should

define the vertex ordering O and the upper bounding function computeUB , to maximize the number of vertices on which the algorithm does not need to branch. We will review the related work on upper bounds and vertex orderings for MaxClique in Section 4. The first call to Algorithm 1 to search for a maximum clique in G is $\text{MC}(G, V, O, \emptyset, \emptyset)$.

Algorithm 1 ($\text{MC}(G, P, O, C, C_{\max})$, a basic BnB algorithm for MaxClique)

Input: $G = (V, E)$, a candidate set $P = \{p_1, p_2, \dots, p_{|P|}\}$ with $p_1 < p_2 < \dots < p_{|P|}$ w.r.t. a vertex ordering O , a growing clique C , the largest clique $C_{\max}$ found so far in G

Output: $C \cup C'$, where C' is a maximum clique of $G[P]$, if $|C \cup C'| > |C_{\max}|$; $C_{\max}$ otherwise

```

1 begin
2   if  $P = \emptyset$  then return  $C$ ;
3   for  $i = |P|$  downto 1 do
4      $P_i \leftarrow \Gamma(p_i) \cap \{p_{i+1}, \dots, p_{|P|}\}$ ;
5      $\text{UB} \leftarrow \text{computeUB}(G, P_i)$ ;
6     /* Compute an upper bound of  $\omega(G[P_i])$  */
7     if  $\text{UB} \geq |C_{\max}| - |C|$  then
8        $C_1 \leftarrow \text{MC}(G, P_i, O, C \cup \{p_i\}, C_{\max})$ ;
9       if  $|C_1| > |C_{\max}|$  then  $C_{\max} \leftarrow C_1$ ;
10  return  $C_{\max}$ .
```

4. Related Work

In this section, we review the related work on upper bounds and vertex orderings, the two most important components of a BnB MaxClique algorithm.

4.1. Upper Bounds Based on Graph Coloring

BnB MaxClique algorithms frequently use heuristic solutions of GCP that can be obtained in reasonable time as an upper bound of $\omega(G)$, based on Proposition 1. We denote an upper bound based on graph coloring or independent set partition by UB_{IS} (for UB based on independent sets).

Fahle (2002) computes an upper bound of $\omega(G)$ by using the constructive heuristic DSATUR to color vertices one by one, and by partitioning in parallel the graph into ISs. The complexity of DSATUR is $O(|V|^3)$.

Régim (2003) uses an upper bound based on a matching algorithm. A matching corresponds to a set of ISs of two vertices here. This upper bound can be computed very quickly in $O(|E|)$, so that it can frequently be used with little cost to filter out vertices that cannot belong to a maximum clique.

MCQ (Tomita and Seki 2003) colors vertices in a predetermined ordering. Suppose that the current ISs are $D_1, D_2, \dots, D_k$ (in this order, k is 0 at the beginning of the coloring process); MCQ inserts the current first vertex v into the first D_i such that v is nonadjacent to all vertices already in D_i . If such a D_i does

not exist, a new independent set D_{k+1} is opened and v is inserted into D_{k+1} . This coloring process has time complexity $O(|V|^2)$, allowing MCQ to obtain the color classes (i.e., the ISs) and the number of color classes at the same time.

MCR (Tomita and Kameda 2007), MCS (Tomita et al. 2010), and MaxCliqueDyn (Konc and Janezic 2007) all use the same coloring process as MCQ to compute the upper bound. In addition, MCS (Tomita et al. 2010) uses a so-called re-number procedure to reduce the number of vertices from which search is necessary. See Algorithm 2 for an implementation of the re-number procedure. The time complexity of the upper bound in MCS is $O(|V|^3)$.

4.2. Upper Bounds Based on MaxSAT Reasoning

Observe that the above upper bounds UB_{IS} cannot be better than $\chi(G)$. In practice, they are often significantly worse than $\chi(G)$, because computing $\chi(G)$ is NP-hard. Note that even $\chi(G)$ may not be a tight upper bound of $\omega(G)$. For example, Figure 1 shows a graph G for which $\chi(G) > \omega(G)$.

A subset of q ISs is said to be *conflicting* if these ISs cannot form a clique of size q . Li and Quan (2010a, b) propose to improve UB_{IS} by detecting conflicts among ISs using MaxSAT reasoning, which can give an upper bound substantially better than UB_{IS} , as will be showed in Proposition 3 later in this section. The approach was originally illustrated in MaxSAT terminology in Li and Quan (2010b) using the graph in Figure 1. We adapt the illustration in graph terminology for reader's convenience.

The graph in Figure 1 can be partitioned into three ISs $\{v_1, v_4, v_6\}$, $\{v_2, v_3\}$, and $\{v_5\}$ using the coloring process of MCQ. This is an optimal partition, giving an upper bound of 3 for $\omega(G)$. Note that the third IS contains only one vertex and is called a *unit IS*. The upper bound of 3 supposes that each IS can contribute a vertex to a maximum clique C . So, v_5 should be in C . Then, v_1, v_6 , and v_2 should not be in C , because they are not adjacent to v_5 . We remove v_1, v_6 , and v_2 from their IS and obtain two new unit ISs: $\{v_4\}$ and $\{v_3\}$. Then, v_4 should be in C , which removes v_3 from its IS and makes its IS empty, because v_3 is not adjacent to v_4 .

Figure 1. A Simple Graph G ($\chi(G) = 3$ and $\omega(G) = 2$)

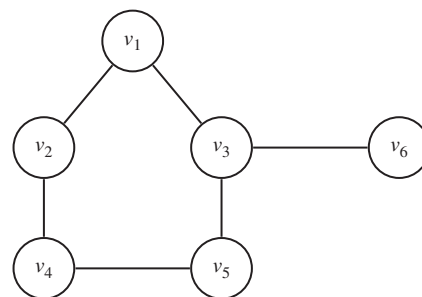
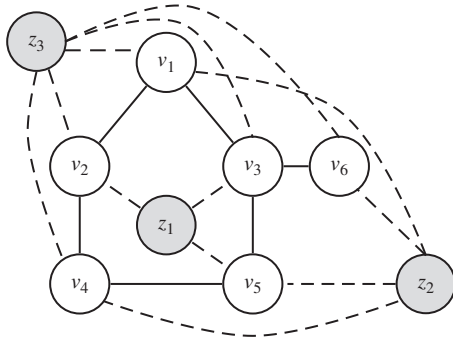


Figure 2. Conflict-Removal Transformation of the Graph of Figure 1

The empty IS means that at least one IS cannot contribute any vertex to C . Consequently, the upper bound of $\omega(G)$ is improved from 3 to 2.

Note that we can easily identify the ISs making the IS $\{v_2, v_3\}$ empty: $\{v_1, v_4, v_6\}$, and $\{v_5\}$. The three ISs are then conflicting. We disable this conflict by adding a new vertex z_j (for $j = 1, 2$, and 3 respectively) into each conflicting IS, such that z_j and $z_{j'}$ are not adjacent for $j \neq j'$, and that each z_j is nonadjacent to any vertex in its IS, but is adjacent to all other vertices in G . We call this operation *conflict-removal transformation* of G . Figure 2 shows the transformed G . Note that the three new ISs $\{v_1, v_4, v_6, z_1\}$, $\{v_2, v_3, z_2\}$ and $\{v_5, z_3\}$ partitioning the transformed G are no longer conflicting, because they form a clique of size 3.

In the general case, let $G_0 = G$ be partitioned into r ISs. MaxSAT reasoning consists in applying the so-called unit IS propagation to identify a set of conflicting ISs $\{D_{11}, D_{12}, \dots, D_{1q_1}\}$ in G_0 . Concretely, MaxSAT reasoning first collects all unit ISs into a stack S , and then repeatedly pops a unit IS $= \{u\}$ from S and removes all vertices nonadjacent to u from their ISs. The ISs becoming unit are pushed into S . The propagation stops when S becomes empty or an IS becomes empty. In the latter case, the conflicting ISs $\{D_{11}, D_{12}, \dots, D_{1q_1}\}$ are identified by retracing the propagation from the empty IS. Then G_0 is transformed into G_1 using the conflict-removal transformation formally defined later.

Definition 1. Let G_{i-1} be partitioned into r ISs, among which $\{D_{i1}, D_{i2}, \dots, D_{iq_i}\}$ are conflicting. The conflict-removal transformation of G_{i-1} into G_i consists in adding a new vertex z_{ij} into D_{ij} ($j = 1, 2, \dots, q_i$) such that

- z_{ij} is not adjacent to any $z_{ij'}$ for $j' \neq j$;
- z_{ij} is not adjacent to any vertex in D_{ij} (so that D_{ij} remains to be an IS after adding z_{ij});
- z_{ij} is adjacent to all other vertices of G_{i-1} .

Note that the r new ISs partition G_1 , in which MaxSAT reasoning applies unit IS propagation to identify a set of conflicting ISs $\{D_{21}, D_{22}, \dots, D_{2q_2}\}$. If such

a conflicting set is identified, G_1 is transformed into G_2 using the conflict-removal transformation. MaxSAT reasoning continues until it fails to detect a conflict in G_h for some $h \geq 0$. Note that h conflicts are disabled in G_h . The following proposition holds.

Proposition 2. $\omega(G_h) = \omega(G_0) + h$.

Proof. We prove the proposition by induction on h . When $h = 0$, the proposition is obviously true. Assume that the proposition is true for $h - 1$ (i.e., $\omega(G_{h-1}) = \omega(G_0) + h - 1$). Let $\{D_{h1}, D_{h2}, \dots, D_{hq_h}\}$ be the set of conflicting ISs detected in G_{h-1} , and let C be a maximum clique of G_{h-1} . There is some j ($1 \leq j \leq q_h$) such that C does not contain any vertex in D_{hj} . Adding z_{hj} into C gives a maximum clique of G_h (note that adding z_{hj} into C excludes all $z_{hj'}$ for $j' \neq j$). So $\omega(G_h) = \omega(G_{h-1}) + 1 = \omega(G_0) + h$. $\square$

Proposition 2 implies the following property.

Proposition 3. Let G be a graph that can be partitioned into r independent sets. If MaxSAT reasoning successively detects h conflicts among these independent sets in such a way that each conflict is disabled by a conflict-removal transformation, then $\omega(G) \leq r - h$.

Proof. Note that G_h can be partitioned into r ISs. So, $\omega(G_h) \leq r$. According to Proposition 2, we have $\omega(G) = \omega(G_h) - h \leq r - h$. $\square$

We denote an upper bound $r - h$ computed using Proposition 3 by UB_{MaxSAT} . The computation consists in repeatedly picking a unit IS $= \{u\}$ to remove all vertices nonadjacent to u from their ISs. The number of vertices nonadjacent to u is in $O(|V|)$, and there are at most $|V|$ unit ISs to pick when detecting a conflict. So, the complexity for detecting a conflict is $O(|V|^2)$ and the complexity for computing UB_{MaxSAT} is $O(h|V|^2)$.

Besides the application in MaxClique solving, the algorithm UB_{MaxSAT} is also shown to be powerful in Li et al. (2012, 2011) for MinSAT (minimum satisfiability) solving.

4.3. Vertex Orderings for BnB MaxClique Algorithms

The vertex ordering for branching (as O in Algorithm 1) in the state-of-the-art BnB MaxClique algorithms such as MCS and BBMC (San Segundo et al. 2011) is dynamically defined at every search tree node. Let $C_{\max}$ be the largest clique found so far in G . MCS first partitions the vertices in G into color classes $D_1, D_2, \dots, D_r$ by assigning the smallest possible color to each vertex. If $r > |C_{\max}|$, the algorithm does not need to branch on vertices in $D_1, \dots, D_{|C_{\max}|}$ because these vertices cannot form a clique larger than $C_{\max}$. It orders the vertices in $D_{|C_{\max}|+1}, \dots, D_r$ according to their color: $v_i < v_j$ if $v_i \in D_p$ and $v_j \in D_q$ and $p > q$. Roughly speaking, vertices with bigger color are smaller in this

ordering. After obtaining the vertex ordering in this way, MCS branches on the vertices from the smallest to the biggest. The original intuition of this ordering for branching is that branching on smaller vertices probably allows us to obtain larger cliques, thus improving $C_{\max}$ more quickly and pruning more subtrees. Recently, McCreesh and Prosser (2014) provided a new explanation for the efficiency of this ordering: Since color classes are filled greedily by placing vertices in the first available color class from left to right, the coloring class D_i with bigger i is likely to be smaller, so that branching first on the vertices in D_i with the biggest i approximates a popular “smallest domain first” heuristic in constraint programming, allowing the reduction of the number of branchings.

The upper bounding procedure in a BnB MaxClique algorithm sequentially colors the vertices in a predetermined ordering. The vertex ordering for coloring is generally simple and is based on vertex degree: the vertices are colored in the nonincreasing order of their degree. In MCS, this ordering is computed in a preprocessing and is stored in a variable named V_a , that is used for coloring at every search tree node. In MCQ, MCR, MCS, and BBMC, the degree of a vertex in a subgraph is not re-computed and the vertex ordering for coloring is always based on the vertex degree in the initial input graph. In MaxCliqueDyn and MaxCLQ (Li and Quan 2010a, b), the degree of a vertex is re-computed dynamically near the root of the search tree to have a more relevant vertex ordering for coloring. Concretely, let $S(\text{level})$ denote the current number of the search tree nodes whose distance from the root is less than or equal to level , and let total denote the total number of the search tree nodes developed so far. MaxCliqueDyn and MaxCLQ recompute the degree of all vertices in the subgraph at a search tree node whose distance from the root is level if $S(\text{level})/\text{total} \leq 0.025$. Then, vertices are re-ordered according to their degree in the subgraph, obtaining better upper bounds near the root of the search tree. The re-computation is performed by simply counting the adjacent vertices for each vertex in the subgraph in MaxCLQ.

Observe that the vertex ordering for coloring and the vertex ordering for branching are usually different in recent state-of-the-art BnB MaxClique algorithms. In this paper, we will propose a new BnB MaxClique algorithm that uses the same vertex ordering for coloring and for branching.

5. Incremental Upper Bound for MaxClique

Let the vertices of a graph $G = (V, E)$ be ordered as $v_1 < v_2 < v_3 < \dots < v_{|V|}$, and the ordering be denoted by O . We define the VertexUB notation as follows.

Definition 2. For any subset $P = \{p_1, p_2, \dots, p_{|P|}\}$ of V such that $p_1 < p_2 < p_3 < \dots < p_{|P|}$ w.r.t. O and any vertex p_i ($1 \leq i \leq |P|$), let $P'_i = \Gamma(p_i) \cap \{v \in P \mid v > p_i \text{ w.r.t. } O\}$.

Then, $\text{VertexUB}[p_i, P, O]$ denotes an upper bound of $\omega(G[\{p_i\} \cup P'_i])$.

In other words, $\text{VertexUB}[p_i, P, O]$ is defined to be an upper bound of the size of the largest clique containing p_i in $G[\{p_i, p_{i+1}, \dots, p_{|P|}\}]$, the subgraph induced by the subset of vertices greater than or equal to p_i w.r.t. O in P .

In Section 5.1, we discuss the property of VertexUB and use VertexUB to define an incremental upper bound called IncUB for a BnB MaxClique algorithm. Note that according to Definition 2, $\text{VertexUB}[p_i, P, O]$ can be assigned any value greater than or equal to $\omega(G[\{p_i\} \cup P'_i])$. A trivial value of $\text{VertexUB}[p_i, P, O]$ can simply be $|P|$. A key issue is how to efficiently make the value of $\text{VertexUB}[p_i, P, O]$ as close to $\omega(G[\{p_i\} \cup P'_i])$ as possible, so that a tight IncUB can be obtained for search pruning. This issue is studied in detail in Section 5.2.

5.1. VertexUB and IncUB

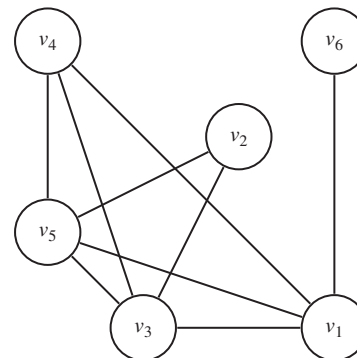
We illustrate VertexUB using the graph in Figure 3.

Example 1. Refer to the graph $G = (V, E)$ in Figure 3. Let the vertex ordering O be: $v_1 < v_2 < v_3 < v_4 < v_5 < v_6$. $\text{VertexUB}[v_2, V, O]$ denotes an upper bound of the size of a maximum clique containing v_2 in $\{v_2, v_3, v_4, v_5, v_6\}$. So, $\text{VertexUB}[v_2, V, O]$ can denote any value greater than or equal to 3, because a maximum clique containing v_2 in $\{v_2, v_3, v_4, v_5, v_6\}$ is $\{v_2, v_3, v_5\}$. Note that $\text{VertexUB}[v_2, V, O]$ must not be any value smaller than 3.

Observe that $\text{VertexUB}[v_2, V, O]$ depends on the vertex ordering O . Let the vertex ordering O' be: $v_6 < v_5 < v_4 < v_3 < v_2 < v_1$. $\text{VertexUB}[v_2, V, O']$ denotes an upper bound of the size of a maximum clique containing v_2 in $\{v_2, v_1\}$. So, it can denote any value greater than or equal to 1. When assigning a value to $\text{VertexUB}[v_2, V, O']$, an algorithm does not need to consider any vertex in $\{v_6, v_5, v_4, v_3\}$, because these vertices are smaller than v_2 w.r.t. O' .

The interest of VertexUB lies in the following property.

Figure 3. A Graph to Illustrate the Incremental Upper Bound



Proposition 4. Let O be a vertex ordering of $G = (V, E)$, and $P = \{p_1, p_2, \dots, p_{|P|}\}$ be a subset of V such that $p_1 < p_2 < p_3 < \dots < p_{|P|}$ w.r.t. O . It holds that, (1) $\omega(G[P]) \leq \max_{p \in P} \text{VertexUB}[p, P, O]$; (2) for any subset $P' \subseteq P$, $\omega(G[P']) \leq \max_{p' \in P'} \text{VertexUB}[p', P, O]$.

Proof. (1) Let C be a maximum clique of $G[P]$ and v be the smallest vertex of C w.r.t. O . Then $\omega(G[P]) = |C| \leq \text{VertexUB}[v, P, O] \leq \max_{p \in P} \text{VertexUB}[p, P, O]$. (2) Let C' be a maximum clique of $G[P']$ and v' be the smallest vertex of C' w.r.t. O . Note that $v' \in P' \subseteq P$ and C' is also a clique of $G[P]$. So, $\omega(G[P']) = |C'| \leq \text{VertexUB}[v', P, O] \leq \max_{p' \in P'} \text{VertexUB}[p', P, O]$. $\square$

Proposition 4 allows us to define an incremental upper bound, denoted by IncUB , of the size of a maximum clique containing p_i in $G[\{p_i, p_{i+1}, \dots, p_{|P|}\}]$.

Definition 3. Let O be a vertex ordering of $G = (V, E)$, and let $P = \{p_1, p_2, \dots, p_{|P|}\}$ be a subset of V such that $p_1 < p_2 < p_3 < \dots < p_{|P|}$ w.r.t. O . Assume that $\text{VertexUB}[p_i, P, O]$ has been computed for each vertex p_i in P . Let $P'_i = \Gamma(p_i) \cap \{p_{i+1}, p_{i+2}, \dots, p_{|P|}\}$. We define

$$\text{IncUB}(p_i, P, O) = \begin{cases} \min(\text{VertexUB}[p_i, P, O], \\ 1 + \max_{u \in P'_i} \text{VertexUB}[u, P, O]) & \text{if } P'_i \neq \emptyset, \\ 1 & \text{otherwise.} \end{cases}$$

To see the meaning of $\text{IncUB}(p_i, P, O)$, note that P'_i is the set of neighbors of p_i in $\{p_{i+1}, p_{i+2}, \dots, p_{|P|}\}$, and that $\max_{u \in P'_i} \text{VertexUB}[u, P, O]$ is an upper bound of the size of a maximum clique C in $G[P'_i]$ according to Proposition 4. Adding p_i to C gives a maximum clique containing p_i in $G[\{p_i, p_{i+1}, \dots, p_{|P|}\}]$, whose size is bounded by $\min(\text{VertexUB}[p_i, P, O], 1 + \max_{u \in P'_i} \text{VertexUB}[u, P, O])$. We say that $\text{IncUB}(p_i, P, O)$ is an incremental upper bound, because it is based on VertexUB previously computed for vertices in P and can in turn be used to improve $\text{VertexUB}[p_i, P, O]$.

In practice, P is a candidate set of vertices to enlarge the current growing clique at a search tree node. When a BnB algorithm has to branch on p_i to search for a maximum clique containing p_i in $G[\{p_i, p_{i+1}, \dots, p_{|P|}\}]$, it can compare $\text{IncUB}(p_i, P, O)$ with the size of the largest clique found so far to see if the branch can be pruned. Nevertheless, the quality of $\text{IncUB}(p_i, P, O)$ depends on the quality of $\text{VertexUB}[p_i, P, O]$ and $\max_{u \in P'_i} \text{VertexUB}[u, P, O]$. We will discuss how to make $\text{VertexUB}[p_i, P, O]$ tight for each vertex p_i in P in the Section 5.2.

5.2. Computing VertexUB

Given a candidate set $P = \{p_1, p_2, \dots, p_{|P|}\}$ of vertices to enlarge the current growing clique at a search tree node, a BnB MaxClique algorithm can use a lower bound r related to the largest clique found so far to

partition P into two subsets A and B , in such a way that vertices in A cannot form any clique of size larger than r . Then, the algorithm branches on every vertex b_i in $B = \{b_1, b_2, \dots, b_{|B|}\}$ to search for a maximum clique containing b_i in $G[\{b_i, b_{i+1}, \dots, b_{|B|}\} \cup A]$. In this paper, we determine A using a coloring process and MaxSAT reasoning, such that all vertices in A are bigger than the vertices in B w.r.t. a vertex ordering O . Meanwhile, we compute $\text{VertexUB}[p_i, P, O]$ by distinguishing three cases: (1) p_i is inserted into A by the coloring process; (2) p_i is inserted into A by MaxSAT reasoning; and (3) $p_i \in B$.

5.2.1. Computing VertexUB for Vertices That Can Be Partitioned Into r ISSs. Algorithm 2 inserts the biggest vertices of P w.r.t. O into r ISSs. The re-number process of MCS is adapted when there are already r ISSs. The algorithm returns B_0 , the set of vertices that cannot be inserted into r ISSs.

We assume that $\text{VertexUB}[p, P, O]$ has been initialized to a reasonable value (see Section 6) for every vertex p in P . When a vertex p_i is inserted into an ISS and B_0 is empty, the number $|\Pi|$ of ISSs formed by $\{p_i, p_{i+1}, \dots, p_{|P|}\}$ is obviously an upper bound of the size of a maximum clique containing p_i in $G[\{p_i, p_{i+1}, \dots, p_{|P|}\}]$. So, $\text{VertexUB}[p_i, P, O]$ is improved into $\min(\text{VertexUB}[p_i, P, O], |\Pi|)$ in line 6, line 10 and line 15. Note that B_0 is empty in line 10, because all vertices so far can be inserted into r or fewer ISSs.

However, when B_0 is not empty, the set $\{p_i, p_{i+1}, \dots, p_{|P|}\}$ contains some vertices (i.e., those vertices in B_0) that cannot be inserted into r independent sets. In this case, $\text{VertexUB}[p_i, P, O]$ keeps its current value and is not improved.

If B_0 is not empty, let $B_0 = \{u_1, u_2, \dots, u_{|B_0|}\}$ such that $u_1 < u_2 < \dots < u_{|B_0|}$ w.r.t. O . Note that $u_{|B_0|}$ is the first vertex inserted into B_0 . Algorithm 2 improves $\text{VertexUB}[p, P, O]$ for each vertex $p \in P$ bigger than $u_{|B_0|}$ w.r.t. O .

5.2.2. Computing VertexUB for Vertices Inserted Into A by MaxSAT Reasoning. Algorithm 2 computes a set Π of r ISSs partitioning $A_0 = P \setminus B_0$. If B_0 is not empty, Algorithm 3 performs MaxSAT reasoning among the ISSs $\Pi \cup \{\{u_{|B_0|}\}, \{u_{|B_0|-1}\}, \dots, \{u_1\}\}$ to detect conflicting ISSs. Here, MaxSAT reasoning is performed incrementally, because the first conflict is detected in $\Pi \cup \{\{u_{|B_0|}\}\}$, followed by the conflict-removal transformation which adds a new vertex into each conflicting ISS as described in Section 4.2, and that the second conflict is detected in $\Pi \cup \{\{u_{|B_0|}\}, \{u_{|B_0|-1}\}\}$ after removing the first conflict, followed by the conflict-removal transformation, and so on. Incremental MaxSAT reasoning stops when it fails to detect a conflict in $\Pi \cup \{\{u_{|B_0|}\}, \{u_{|B_0|-1}\}, \dots, \{u_i\}\}$ for some $i \leq |B_0|$ after removing the previous conflicts.

Algorithm 2 (FilterByColoring(G, P, r, O), for a BnB algorithm searching for a maximum clique containing more than r vertices in P)

Input: a graph G , a candidate set $P = \{p_1, p_2, \dots, p_{|P|}\}$, an integer r and a vertex ordering $O: p_1 < p_2 < \dots < p_{|P|}$.
VertexUB[p, P, O] is initialized for every vertex p in P

Output: a set B_0 of branching vertices

```

1 begin
2    $B_0 \leftarrow \emptyset; \Pi \leftarrow \emptyset;$  /*  $\Pi$  will be a set of ISs */
3   for  $i = |P|$  downto 1 do
4     if  $\Pi$  contains an IS  $D$  in which  $p_i$  is not adjacent to any vertex then
5        $D \leftarrow D \cup \{p_i\};$  /* update  $D$  in  $\Pi$  */
6       if  $B_0 = \emptyset$  then VertexUB[ $p_i, P, O$ ]  $\leftarrow \min(\text{VertexUB}[p_i, P, O], |\Pi|);$ 
7     else
8       if  $|\Pi| < r$  then
9         create a new IS  $D = \{p_i\}; \Pi \leftarrow \Pi \cup \{D\};$ 
10        VertexUB[ $p_i, P, O$ ]  $\leftarrow \min(\text{VertexUB}[p_i, P, O], |\Pi|);$ 
11      else
12        /* Re-NUMBER of MCS */
13        if  $\Pi$  contains an IS  $D$  in which  $p_i$  has only one adjacent vertex  $u$ ,
14          and  $u$  can be inserted into another IS  $D'$  then
15           $D' \leftarrow D' \cup \{u\}; D \leftarrow (D \setminus \{u\}) \cup \{p_i\};$  /* update  $D'$  and  $D$  in  $\Pi$  */
16          if  $B_0 = \emptyset$  then VertexUB[ $p_i, P, O$ ]  $\leftarrow \min(\text{VertexUB}[p_i, P, O], |\Pi|);$ 
17        else
18          insert  $p_i$  into  $B_0;$ 
19   return  $B_0$ .
```

Then it holds that $\omega(G[A_0 \cup \{u_{|B_0|}, u_{|B_0|-1}, \dots, u_{i+1}\}]) \leq r$ according to Proposition 3, because the set $A_0 \cup \{u_{|B_0|}, u_{|B_0|-1}, \dots, u_{i+1}\}$ can be partitioned into $r + |B_0| - i$ ISs and $|B_0| - i$ conflicts are detected among these ISs.

So, incremental MaxSAT reasoning determines two sets $A = \{a \in P \mid a > u_i \text{ w.r.t. } O\}$ and $B = \{b \in P \mid b \leq u_i \text{ w.r.t. } O\}$. Note that $A \subseteq A_0 \cup \{u_{|B_0|}, u_{|B_0|-1}, \dots, u_{i+1}\}$, because A_0 can contain some vertices smaller than u_i . Therefore, A cannot form any clique containing more than r vertices. Clearly, for each $a \in A$ and each $b \in B$, it holds $a > b$ w.r.t. O .

Meanwhile, Algorithm 3 improves VertexUB[a, P, O] to $\min(r, \text{VertexUB}[a, P, O])$ for each vertex a such that $u_{|B_0|} \geq a > u_i$ (line 19). Note that VertexUB[a, P, O] has been already improved for each vertex $a \in A$ such that $a > u_{|B_0|}$ in Algorithm 2. So, VertexUB[a, P, O] is improved for each $a \in A$ after running Algorithms 2 and 3.

Finally Algorithm 3 returns $B = P \setminus A = \{b \in P \mid b \leq u_i \text{ w.r.t. } O\}$, a set of vertices on which a BnB MaxClique algorithm will branch to search for a maximum clique containing more than r vertices in P . VertexUB[b, P, O] will be improved for each $b \in B$ after branching on b .

Incremental MaxSAT reasoning was originally proposed in Li et al. (2015) for MaxClique. Its combination with an incremental upper bound is new in this paper. Moreover, its soundness is formally proved here for the first time.

5.2.3. Computing VertexUB for the Branching Vertices. Given a candidate P and a partition of P into two sets A and B , where $B = \{b_1, b_2, \dots, b_{|B|}\}$ such that

$b_1 < b_2 < \dots < b_{|B|}$ and $b_{|B|}$ is smaller than all vertices in A w.r.t. O , a BnB MaxClique algorithm will branch on each vertex b_i for $i = |B|, |B| - 1, \dots, 1$ to search for a maximum clique containing b_i in $\{b_i, b_{i+1}, \dots, b_{|B|}\} \cup A$. The obtained maximum clique can be used to improve VertexUB[b_i, P, O], because $\{b_i, b_{i+1}, \dots, b_{|B|}\} \cup A$ is equal to $\{v \in P \mid v \geq b_i\}$ (see Section 6).

6. IncMC2: A BnB MaxClique Algorithm with Incremental Upper Bound

Algorithm 4 shows a BnB MaxClique solver called IncMC2 that exploits the incremental upper bound IncUB. Given a candidate set P with an ordering O in which vertexUB[p, P, O] has been initialized for every vertex p , the current growing clique C in which every vertex is adjacent to every vertex in P , and the largest clique $C_{\max}$ found so far in G , IncMC2 tries to enlarge C by inserting the vertices of P into C . For this purpose, it first calls Algorithm 2 to partition P into B_0 and $A_0 = P \setminus B_0$ in such a way that vertices in A_0 are partitioned into $|C_{\max}| - |C|$ ISs. If B_0 is empty, a clique of size larger than $|C_{\max}| - |C|$ cannot be found in $G[P]$, IncMC2 prunes the search and returns $C_{\max}$.

If B_0 is not empty, let the biggest vertex w.r.t. O in B_0 be $u_{|B_0|}$. Note that A_0 can contain some vertices smaller than $u_{|B_0|}$ w.r.t. O , and that VertexUB[p, P, O] is improved for each vertex p in A_0 such that $p > u_{|B_0|}$ w.r.t. O . IncMC2 calls Algorithm 3 to reduce B_0 and returns a set B of branching vertices. If B is empty, IncMC2 prunes the search and returns $C_{\max}$, because

Algorithm 3 (IncMaxSAT(G, P, O, A_0, B_0), incremental MaxSAT reasoning to reduce B_0)

Input: a graph $G = (V, E)$ and an ordering O over V , a candidate set P partitioned into A_0 and B_0 .
 A_0 can be partitioned into r ISs, $B_0 = \{u_1, u_2, \dots, u_{|B_0|}\}$ such that $u_1 < u_2 < \dots < u_{|B_0|}$ w.r.t. O

Output: a set of branching vertices

```

1 begin
2   partition  $A_0$  into  $r$  ISs and store the  $r$  ISs in  $\Pi$ ;
3   for  $i = |B_0|$  downto 1 do
4     push the unit IS  $\{u_i\}$  into  $\Pi$  and into an empty stack  $S$ ;
5     while  $S$  is not empty do
6       pop a unit IS  $\{u\}$  from  $S$ ;
7       remove all vertices nonadjacent to  $u$  from the ISs of  $\Pi$ ;
8       for each new unit IS  $D$  do
9         push  $D$  into  $S$ ;
10      if there is an IS  $D$  that becomes empty then
11        restore all removed vertices into their independent set;
12        /* conflict-removal transformation, see Definition 1 in Section 4.2 */
13        transform  $G$  by adding a new vertex in  $D$  and in each IS responsible for making  $D$  empty;
14        break;
15      if no empty IS is derived then break;
16  if  $i = 0$  then return  $\emptyset$ ; /* a conflict has been derived for every vertex in  $B_0$  in this case */
17  /*  $u_i$  is the biggest vertex in  $B_0$  for which IncMaxSAT fails to derive a conflict */
18  for each vertex  $a \in P$  such that  $u_{|B_0|} \geq a > u_i$  do
19     $\text{VertexUB}[a, P, O] \leftarrow \min(\text{VertexUB}[a, P, O], r)$ ;
20 return the set  $\{p \in P \mid p \leq u_i\}$ .
```

incremental MaxSAT reasoning proves that P cannot contain any clique larger than $|C_{\max}| - |C|$.

If B is not empty, let $B = \{b_1, b_2, \dots, b_{|B|}\}$ such that $b_1 < b_2 < \dots < b_{|B|}$ w.r.t. O and $A = P \setminus B$. IncMC2 does not need to branch on vertices in A thanks to Algorithm 2 and Algorithm 3, and only has to branch on each b_i for $i = |B|, |B| - 1, \dots, 1$ to search for a maximum clique containing b_i and more than $|C_{\max}| - |C|$ vertices in $G[\{b_i, b_{i+1}, \dots, b_{|B|}\} \cup A]$. Note that for any $a \in A$, $a > b_{|B|}$ w.r.t. O and $\text{VertexUB}[a, P, O]$ has been improved by Algorithm 2 and Algorithm 3.

Before branching on b_i , IncMC2 computes the value $\text{IncUB}(b_i, P, O)$ in linear time according to Definition 3. If $\text{IncUB}(b_i, P, O) \leq |C_{\max}| - |C|$, the branch is pruned and $\text{VertexUB}[b_i, P, O]$ is improved to $\text{IncUB}(b_i, P, O)$ (line 10). Otherwise, the branch cannot be pruned, and the neighbors of b_i in $\{b_{i+1}, \dots, b_{|B|}\} \cup A$ constitute the new candidate set P'_i for the branch, for which $\text{VertexUB}[p', P'_i, O]$ is initialized to $\text{VertexUB}[p', P, O]$ for each $p' \in P'_i$. The branch on b_i returns C_1 which is either a maximum clique larger than $C_{\max}$ or $C_{\max}$ if such a clique does not exist. Consequently, $\text{VertexUB}[b_i, P, O]$ is improved to $|C_1| - |C|$ (line 16).

Note that before calling IncMC2 to search for a maximum clique in $G = (V, E)$, we have to define a vertex ordering O , which can be a simple nondecreasing degree ordering or a more sophisticated ordering as described in Section 7. Then, $\text{VertexUB}[v, V, O]$ should be initialized for each vertex v in V . Let $V = \{v_1, v_2, \dots, v_{|V|}\}$ such that $v_1 < v_2 < \dots < v_{|V|}$ w.r.t. O . For

Algorithm 4 (IncMC2($G, P, O, C, C_{\max}$), a new BnB algorithm for MaxClique)

Input: $G = (V, E)$, a candidate set P , a vertex ordering O over V , the current growing clique C , the largest clique $C_{\max}$ found so far in G

Output: $C \cup C'$, where C' is a maximum clique of $G[P]$, if $|C \cup C'| > |C_{\max}|$; $C_{\max}$ otherwise

```

1 begin
2   if  $P = \emptyset$  then return  $C$ ;
3    $B_0 \leftarrow \text{FilterByColoring}(G, P, |C_{\max}| - |C|, O)$ ;
   /* Algorithm 2 */
4   if  $B_0 = \emptyset$  then return  $C_{\max}$ ;
5    $B \leftarrow \text{IncMaxSAT}(G, P, O, P \setminus B_0, B_0)$ ;
   /* Algorithm 3 */
6   if  $B = \emptyset$  then return  $C_{\max}$ ;
7    $A \leftarrow P \setminus B$ ;
8   for  $i = |B|$  downto 1 do
9     if  $\text{IncUB}(b_i, P, O) \leq |C_{\max}| - |C|$  then
10       $\text{VertexUB}[b_i, P, O] \leftarrow \text{IncUB}(b_i, P, O)$ ;
11    else
12       $P'_i \leftarrow \Gamma(b_i) \cap (\{b_{i+1}, \dots, b_{|B|}\} \cup A)$ ;
13      foreach  $p' \in P'_i$  do
14         $\text{VertexUB}[p', P'_i, O] \leftarrow \text{VertexUB}[p', P, O]$ ;
15       $C_1 \leftarrow \text{IncMC2}(G, P'_i, O, C \cup \{b_i\}, C_{\max})$ ;
16       $\text{VertexUB}[b_i, P, O] \leftarrow |C_1| - |C|$ ;
17      if  $|C_1| > |C_{\max}|$  then  $C_{\max} \leftarrow C_1$ ;
18 return  $C_{\max}$ .
```

$i = |V|, |V| - 1, \dots, 1$, $\text{VertexUB}[v_i, V, O]$ is first initialized to the degree of v_i plus 1, then to $\text{IncUB}(v_i, V, O)$ computed using Definition 3.

After defining O and initializing $\text{VertexUB}[v, V, O]$ for each v , we call $\text{IncMC2}(G, V, O, \emptyset, \emptyset)$ to search for a maximum clique in G .

See the online supplement for an illustration of the use of VertexUB and IncUB to prune search space.

7. Vertex Ordering in IncMC2

The ordering for branching in an exact algorithm is a very important issue for solving NP-hard problems such as SAT, CSP, GCP, and MaxClique. In IncMC2, as well as in IncMaxCLQ, the vertex ordering for branching should allow to minimize the initial value of $\text{VertexUB}[v, V, O]$ for each vertex v . Refer to Figure 1. If vertices are ordered as $O: v_1 < v_2 < v_3 < v_4 < v_5 < v_6$, $\text{VertexUB}[v_i, V, O]$ would be initialized respectively to 4, 3, 2, 2, 1, 1 for $i = 1, 2, 3, 4, 5, 6$ using Definition 3. However, if vertices are ordered as $O': v_5 < v_2 < v_3 < v_1 < v_4 < v_6$, $\text{VertexUB}[v_i, V, O']$ would be initialized respectively to 3, 2, 2, 1, 1, 1 for $i = 5, 2, 3, 1, 4, 6$ using Definition 3. The ordering O' is thus better than O , because it better captures the graph structure.

Observe that, in the second ordering O' , the last three vertices $\{v_1, v_4, v_6\}$ constitute a maximum independent set of the graph, allowing us to maximize the number of vertices v such that $\text{VertexUB}[v, V, O'] = 1$, and the two preceding vertices v_2 and v_3 constitute the second maximum independent set of the graph (after removing the first one), maximizing the number of vertices having the VertexUB value of 2. Based on this observation, we propose the *MIS* (short for Maximum Independent Set) *vertex ordering*: let $D_1, D_2, \dots, D_r$ be successive maximum independent sets partitioning G , $v_i < v_j$ iff $v_i \in D_p$ and $v_j \in D_q$ such that $p > q$ or $p = q$ and the degree of v_i is smaller than the degree of v_j . Note that $|D_p| \leq |D_q|$ if $p > q$.

Although capturing well the graph structure, the MIS vertex ordering has two drawbacks, as follows

- Searching for a maximum independent set (MIS) is generally hard in a sparse graph. We introduce a parameter α ($0 < \alpha < 1$) and do not compute the MIS ordering for any graph with a density smaller than or equal to α . The real value of α is empirically determined according to the performance of the BnB algorithm for finding a MIS in a graph with density greater than α . In this paper, we use $\alpha = 0.6$, because IncMC2 computes the MIS ordering in less than one second for all graphs with density greater than 0.6 considered in this paper, which is negligible compared with the whole solving time.

- If the density of G is higher than α , we can easily obtain the successive MISs $D_1, D_2, \dots, D_r$ partitioning G . The partition is considered to be *regular*, if there

are at most β MISs of cardinality 1, where β is an integer parameter. We apply the MIS vertex ordering if the partition is regular, because the MIS ordering makes sense only if most MISs partitioning G contain more than one vertex.

When the MIS vertex ordering is not used, we use an ordering $v_1 < v_2 < v_3 < \dots < v_{|V|}$ computed as follows: v_1 is the vertex with the smallest degree in G , v_2 is the vertex with the smallest degree in G after removing v_1 , and so on. This ordering was first used in Carraghan and Pardalos (1990) for branching in a BnB MaxClique algorithm. We call it the *Deg* ordering in this paper.

Algorithm 5 (VertexOrdering(G))

Input: A graph $G = (V, E)$

Output: An ordering of V

```

1 begin
2   sort the vertices of  $G$  in the Deg ordering
    $v_1 < v_2 < \dots < v_n$ ;
3   if the density of  $G$  is lower than or equal to 0.6 then
4     return the Deg ordering;
5   else
6     partition  $G$  into  $r$  ISs by searching for
       successive maximum cliques in  $\bar{G}$  using
       IncMC2 with the inverse Deg
       ordering  $v_n < v_{n-1} < \dots < v_1$ ;
7     compute an initial clique  $C$  of  $G$ ;
8     if there are at most three MISs of cardinality 1
       or  $r < 1.1 \times |C|$  then
9       return the MIS vertex ordering;
10    else
11      return the Deg ordering
        $v_1 < v_2 < \dots < v_n$ ;

```

The MIS ordering and the automatic switching between the MIS ordering and the Deg ordering were originally shown effective in IncMaxCLQ to solve MaxClique (Li et al. 2013), which are also adopted in the newest version of BBMC (San Segundo et al. 2016). In IncMaxCLQ, $\alpha = 0.7$ and $\beta = 2$. In IncMC2, $\alpha = 0.6$ and $\beta = 3$, because IncMC2 is more efficient to search for a MIS in a graph with density > 0.6 and we want to favor more the MIS ordering. In addition, IncMC2 searches for an initial clique $C = \{v_1, v_2, \dots\}$ in G as follows: select the vertex v_1 with the biggest degree and remove all vertices nonadjacent to v_1 , select v_2 with the biggest degree in the remaining vertices and remove all vertices nonadjacent to v_2 , and so on. Let r be the number of MISs partitioning G . If $r < \eta \times |C|$, then IncMC2 uses the MIS ordering, where η is empirically fixed to 1.1. Algorithm 5 summarizes the vertex ordering used in IncMC2.

8. Experimental Analysis

We implemented IncMC2 in C and compiled it using GNU gcc -O3. In the implementation, vertices are represented by integers, and we use standard integer

arrays for everything in the algorithms. For example, we use integer arrays to store the list of neighbors and the list of nonneighbors of each vertex, the ISs, VertexUB, and the stacks to control backtracking. This implementation might be improved using special data structures such as bit strings used in BBMC.

We conducted experiments to compare IncMC2 with other BnB algorithms in the literature and to analyze the effect of IncUB on the performance of IncMC2. The experiments were performed on an Intel Xeon CPU X5460@3.16 GHz under Linux with 16 GB memory and 12 MB of cache memory (shared by eight running cores). The dfmax program (compiled using GNU gcc-O3) needs 0.18, 1.04, and 3.95 seconds to solve the DIMACS benchmark graphs r300.5, r400.5, and r500.5, respectively, on our machine.

We first describe the benchmarks used and the solvers compared, then discuss and analyze the experimental results.

8.1. Benchmarks

MaxClique solvers are proposed in the literature usually by showing their performance on random and/or DIMACS graphs¹ (San Segundo et al. 2011; Tomita and Seki 2003; Tomita and Kameda 2007; Tomita et al. 2010; Konc and Janezic 2007; Östergård 2002; Carraghan and Pardalos 1990; Fahle 2002; Régim 2003; Li and Quan 2010a, b). DIMACS graphs fall into three categories: graphs generated using randomness and some properties, crafted graphs with special structures, and graphs from real-world problems.

More recently, it is recommended in Cai et al. (2011), Grosso et al. (2008) to use the *frb** graphs (BHOSLIB²) to evaluate MaxClique and MVC (Minimum Vertex Cover) solvers. The *frb** graphs are encoded from CSP instances generated using the CSP model RB (Xu et al. 2007, Xu and Li 2000). They are often used to evaluate heuristic MaxClique and MVC solvers (Grosso et al. 2008, Pullan et al. 2011, Cai et al. 2011, Benlic and Hao 2013), because they are very hard for exact solvers. As indicated in Prosser (2012), state-of-the-art exact solvers can only solve the few smallest and easiest *frb** graphs in reasonable time.

In this paper, we use random graphs, DIMACS graphs and *frb** graphs in our experiments, which are essential benchmarks used in Prosser (2012), Wu and Hao (2015) to study MaxClique solvers.

8.2. Solvers

We first compare IncMC2 with the following BnB MaxClique algorithms to assess its performance:

- MaxCLQ. We use the version presented in Li and Quan (2010a).
- IncMaxCLQ. IncMaxCLQ is written as IncMC in the tables of this section. At every search tree node, IncMaxCLQ computes three upper bounds IncUB,

UB_{IS}, and UB_{MaxSAT} to prune branches, while IncMC2 exploits IncUB differently.

- MaxCliqueDyn³, version Dec. 2012. It is improved from the 2010 version which was better than MCR (Li and Quan 2010b).

- MCS (Tomita et al. 2010) (improved from MCR). The original codes of MCS are not open. We use a faithful implementation by Wang Kai following the description in Tomita et al. (2010), which has a similar performance to the original version (Wang 2013). Wang Kai uses the coloring repair step as described in Tomita et al. (2010), but does not use a bitset encoding as BBMC.

- BBMC (San Segundo et al. 2011). We use the last release of BBMC kindly provided by Pablo San Segundo in July 2016, which uses the similar automatic initial vertex ordering as IncMC2 and IncMaxCLQ (San Segundo et al. 2016).

Then, we compare the following solvers to identify the effect of IncUB on the performance of IncMC2. Note that the solvers are named with the upper bound(s) used in them.

- IncMC2, also denoted as UB_{MaxSAT} + IncUB to emphasize the difference with other solvers.

- UB_{MaxSAT}, identical to IncMC2 except that it does not use IncUB to prune branches. In other words, UB_{MaxSAT} is IncMC2 except that lines 9 and 10 are removed in Algorithm 4.

- UB_{IS} + IncUB, identical to IncMC2 except that it only uses graph coloring to partition the candidate set P into A and B . Concretely, UB_{IS} + IncUB is IncMC2 except that lines 5 and 6 calling IncMaxSAT in Algorithm 4 are replaced by $B \leftarrow \{u \in P \mid u \leq u_{|B_0|} \text{ w.r.t. } O\}$, where $u_{|B_0|}$ is the biggest vertex w.r.t. O in B_0 obtained in line 3, in order to guarantee that any vertex in B is smaller than any vertex in A w.r.t. O , which is a necessary condition to use IncUB.

- UB_{IS}, identical to UB_{IS} + IncUB except that it does not use IncUB to prune branches. Concretely, UB_{IS} is UB_{IS} + IncUB except that lines 9 and 10 calling IncUB for pruning branches in Algorithm 4 are removed.

UB_{MaxSAT} + IncUB, UB_{MaxSAT}, UB_{IS} + IncUB, and UB_{IS} all use the vertex ordering automatically computed by Algorithm 5 for independent set partition and for branching.

All our solvers are available at www.mis.u-picardie.fr/~cli/programs-for-joc.zip.

8.3. Comparison of IncMC2 with Other BnB MaxClique Solvers

Table 1 compares IncMC2 with MaxCliqueDyn (Dyn in the table), MaxCLQ, IncMaxCLQ (IncMC in the table), MCS, and BBMC on random graphs. For each number of vertices and each density, we generate 51 graphs to be solved using each solver. We say that a graph is

Table 1. Mean Runtimes (in Seconds) of the Compared Solvers to Solve Random Graphs with Different Numbers of Vertices (N) and Different Densities (D)

N	D	$\omega_{\max}$	$\omega_{\min}$	ω_{avg}	ω_{med}	Dyn	MCS	MaxCLQ	IncMC	BBMC	IncMC2
200	0.90	43	40	41.2	41	65.34	37.61	9.55	6.56	75.64	5.95
200	0.95	65	59	62.4	62	34.84	23.76	1.69	0.95	12.85	0.90
300	0.80	30	27	28.5	28	270.0	209.6	116.3	119.7	127.2	130.7
300	0.90	50	47	48.0	48	—	—	3,951	3,576	4,093	3,110
#solved						0	0	4	20	1	40
400	0.70	22	21	21.5	21	98.92	94.06	87.17	84.03	56.72	87.61
400	0.80	32	30	30.5	31	—	—	4,175	3,332	4,548	3,016
#solved						0	0	28	21	23	32
500	0.60	18	16	17.0	17	26.26	23.30	30.60	27.11	16.67	16.52
500	0.70	23	22	22.5	22	1,201	915.5	763.4	877.0	529.9	808.0
1,000	0.30	10	9	9.5	9	0.73	0.67	2.31	1.12	14.28	0.90
1,000	0.40	13	12	12.0	12	8.54	7.59	19.42	9.24	17.70	6.59
1,000	0.50	16	15	15.0	15	175.6	166.0	300.2	215.1	130.0	137.9
2,000	0.20	9	8	8.1	8	1.15	1.08	7.92	2.53	42.43	1.45
2,000	0.30	11	10	10.6	11	23.98	20.59	147.0	31.92	51.58	21.07
2,000	0.40	14	13	13.1	13	622.9	611.6	1,910	891.6	633.9	590.1
5,000	0.10	7	7	7.0	7	6.81	5.88	56.81	19.55	120.5	5.70
5,000	0.20	10	9	9.1	9	114.1	109.1	2,330	422.5	243.1	97.22
10,000	0.10	8	7	7.2	7	162.1	144.8	1,235	269.7	423.2	52.56
10,000	0.20	11	10	10.0	10	3,421	3,283	—	4,220	4,940	3,344
#solved						3	3	0	1	2	51
15,000	0.10	8	8	8.0	8	750.5	970.5	—	1,105	1,093	299.1
20,000	0.05	7	6	6.2	6	341.5	251.7	—	190.6	1,591	165.9

Note. Boldface numbers stand for the best runtime in each line.

solved by a solver if the solver finds and proves a maximum clique within a cutoff time. To make the comparison clearer, we exclude the easy points in which the mean runtime is less than 10 seconds for all compared solvers, and the hard points in which no graph is solved by any solver within 5,000 seconds. The displayed mean runtimes are computed among the solved graphs. When a solver cannot solve any graph at a point, its mean runtime is marked with “—”. If there is a solver that solves fewer than 51 graphs within 5,000 seconds at a point, the number of solved graphs (#solved) by each solver at the point is shown in the next row. We also give the max, min, average, and median clique number of the graphs at each point, denoted by $\omega_{\max}$, $\omega_{\min}$, ω_{avg} , and ω_{med} , respectively.

We say that solver 1 is significantly faster than solver 2 at a point if solver 1 solves more graphs than solver 2, or the mean runtime of solver 1 is at least two times shorter than the mean runtime of solver 2. We first look at the three solvers exploiting MaxSAT reasoning: MaxCLQ, IncMaxCLQ and IncMC2. The main difference between MaxCLQ and IncMaxCLQ is that IncMaxCLQ uses, in addition, IncUB to prune search. However, IncMaxCLQ is significantly faster than MaxCLQ at 12 points, while MaxCLQ is significantly faster than IncMaxCLQ at only one point: (400,0.8). With a new combination of IncUB and MaxSAT reasoning, IncMC2 is generally faster than IncMaxCLQ and is significantly faster than IncMaxCLQ at seven points, while IncMaxCLQ is never significantly faster than IncMC2.

IncMC2 is also generally faster than MCS and is significantly faster than MCS at seven points, while MCS is never significantly faster than IncMC2. BBMC is faster than IncMC2 at four points but is never significantly faster than IncMC2. However, IncMC2 is faster than BBMC at 16 points and is significantly faster than BBMC at 14 points, including small dense graphs and large sparse graphs.

Table 2 compares IncMC2 with MaxCliqueDyn (Dyn in the table), MaxCLQ, IncMaxCLQ (IncMC in the table), MCS, and BBMC on DIMACS graphs that are solved by at least one solver within 10^5 seconds, excluding the graphs solved by all solvers within one second. In the table, “—” stands for instances that cannot be solved within the cutoff time 10^5 seconds.

IncMC2 and IncMaxCLQ are the two only solvers that are able to solve all the graphs in the table thanks to IncUB and MaxSAT reasoning. IncMC2 is faster than IncMaxCLQ for nearly all graphs and is significantly faster than IncMaxCLQ for nine graphs. IncMC2 is faster or significantly faster than MCS for all graphs except p_hat1500-1 and hamming10-2. Compared with BBMC, IncMC2 is faster or significantly faster for all graphs except brock400_2 and brock400_4.

Table 3 compares IncMC2 with MaxCliqueDyn (Dyn in the table), MaxCLQ, IncMaxCLQ (IncMC in the table), MCS, and BBMC on BHOSLIB graphs that are solved by at least one solver in 3,600 seconds. The graphs that cannot be solved within 3,600 seconds are marked by “—”. Note that the clique number of a graph named “frbd- x - y ” is d .

Table 2. Runtimes (in Seconds) for DIMACS Graphs

Instance	ω	N	D	Dyn	MCS	MaxCLQ	IncMC	BBMC	IncMC2
brock400_1	27	400	0.74	466.0	379.7	339.2	222.3	210.2	131.7
brock400_2	29	400	0.74	192.1	166.2	105.9	170.2	78.09	97.44
brock400_3	31	400	0.74	371.6	256.2	102.4	204.6	111.4	73.69
brock400_4	33	400	0.74	185.7	138.2	125.3	159.2	13.66	20.22
brock800_1	23	800	0.65	5,988	5,209	4,889	8,830	3,572	1,862
brock800_2	24	800	0.65	5,349	4,686	4,857	11,210	3,000	1,891
brock800_3	25	800	0.65	3,455	3,208	3,452	4,221	1,674	737.2
brock800_4	26	800	0.65	2,691	2,259	3,441	5,832	1,391	1,300
C2000.5	16	2,000	0.50	—	—	—	61,009	46,875	37,401
C250.9	44	250	0.89	2,376	2,074	298.2	278.9	889.0	189.4
DSJC1000.5	15	1,000	0.50	185.2	169.6	295.8	226.3	135.2	126.5
gen400_p0.9_55	55	400	0.90	—	37,220	—	1.23	1.89	1.09
gen400_p0.9_65	65	400	0.90	—	96,567	26,134	0.34	1.49	0.32
gen400_p0.9_75	75	400	0.90	—	—	1,372	0.27	1.48	0.21
hamming10-2	512	1,024	0.99	49.73	0.19	0.06	32.65	357.1	35.46
keller5	27	776	0.75	—	—	5,376	141.6	2,201	180.0
MANN_a27	126	378	0.99	2.81	0.36	0.12	0.34	6.03	0.25
MANN_a45	345	1,035	0.99	1,712	63.09	20.04	115.8	157.3	10.51
p_hat1000-2	46	1,000	0.49	276.6	131.6	219.9	48.75	51.49	25.69
p_hat1000-3	68	1,000	0.75	—	—	—	42,244	—	22,979
p_hat1500-1	12	1,500	0.25	2.84	2.38	9.75	5.32	15.53	3.32
p_hat1500-2	65	1,500	0.51	—	10,448	15,138	2,165	3,625	865.9
p_hat300-3	36	300	0.74	3.06	1.43	1.13	0.37	1.84	0.32
p_hat500-2	36	500	0.50	1.04	0.42	0.66	0.19	2.19	0.28
p_hat500-3	50	500	0.75	235.1	79.23	81.04	22.02	30.20	13.82
p_hat700-2	44	700	0.49	8.60	3.33	4.61	0.92	3.92	1.06
p_hat700-3	62	700	0.75	3,946	1,586	1,009	269.5	398.7	154.6
san400_0.7_3	22	400	0.70	1.50	0.77	3.1	1.91	2.57	0.20
sanr200_0.9	42	200	0.90	32.56	19.68	6.08	2.71	7.83	1.84
sanr400_0.7	21	400	0.70	110.0	99.69	98.56	104.7	67.63	61.35

Note. Boldface numbers stand for the best runtime in each line.

BBMC, IncMaxCLQ, and IncMC2 are the three best solvers for BHOSLIB graphs, which all automatically select the *MIS* ordering. However, BBMC does not use incremental MaxSAT reasoning and IncUB. IncMaxCLQ uses IncUB, but MaxSAT reasoning is not incremental in IncMaxCLQ. Thanks to a novel combination of IncUB and incremental MaxSAT reasoning, IncMC2 is faster than BBMC for 26 BHOSLIB graphs and is regularly faster than IncMaxCLQ for all BHOSLIB graphs, while BBMC is faster than IncMC2 only for three graphs.

In summary, IncMC2 is globally the fastest solver in Tables 1–3. Recall that IncMC2 is a single thread solver that does not use any special data structure such as bit-string. It might be parallelized using the low-overhead, scalable work splitting mechanism proposed in McCreesh and Prosser (2015) to solve these graphs in shorter time.

8.4. Effect of IncUB on the Performance of IncMC2

Tables 4–6 compare solvers UB_{IS} , $UB_{IS} + IncUB$, UB_{MaxSAT} , and $UB_{MaxSAT} + IncUB$ (i.e., IncMC2) in terms of runtimes and search tree sizes, respectively, on random, DIMACS, and BHOSLIB graphs. The gain with

IncUB for a graph is computed by $(\text{withoutIncUB} - \text{withIncUB}) / \text{withIncUB} \times 100\%$ and is averaged in the last line.

Table 4 uses the same 51 random graphs as Table 1 at each point. When a solver cannot solve any graph within 5,000 seconds, the entry is marked with “—”. When a solver solves fewer than 51 graphs within 5,000 seconds at a point, the number of solved graphs (#solved) by each solver is shown in the next row. $UB_{MaxSAT} + IncUB$ and $UB_{IS} + IncUB$ are regularly better than the version without IncUB both in terms of search tree size and in terms of mean runtime.

Note that the gain in terms of mean runtime is more important than the gain in terms of search tree size. In fact, IncUB is computed in linear time. If a branch is not pruned by IncUB, then independent set partition and/or incremental MaxSAT reasoning have to be performed in cubic time in the recursive call of IncMC2. The use of IncUB in $UB_{MaxSAT} + IncUB$ and $UB_{IS} + IncUB$ allows the two solvers to avoid many calls to the more expensive independent set partition and/or incremental MaxSAT reasoning.

The gain thanks to IncUB is more important for DIMACS graphs than for random graphs, as shown

Table 3. Runtimes (in Seconds) for BHOSLIB Graphs

Instance	ω	N	D	Dyn	MCS	MaxCLQ	IncMC	BBMC	IncMC2
frb30-15-1	30	450	0.82	2,500	992.8	493.6	0.13	2.20	0.12
frb30-15-2	30	450	0.82	—	1,432	849.2	0.15	2.19	0.14
frb30-15-3	30	450	0.82	2,741	766.5	365.5	0.21	2.21	0.20
frb30-15-4	30	450	0.82	—	2,620	1,058	0.21	2.19	0.20
frb30-15-5	30	450	0.82	—	1,109	663.5	0.16	2.18	0.15
frb35-17-1	35	595	0.84	—	—	—	0.89	3.17	0.80
frb35-17-2	35	595	0.84	—	—	—	1.27	3.19	1.14
frb35-17-3	35	595	0.84	—	—	—	0.35	3.21	0.30
frb35-17-4	35	595	0.84	—	—	—	0.38	3.43	0.33
frb35-17-5	35	595	0.84	—	—	—	0.53	3.45	0.47
frb40-19-1	40	760	0.85	—	—	—	1.62	5.09	1.44
frb40-19-2	40	760	0.85	—	—	—	0.52	—	0.44
frb40-19-3	40	760	0.85	—	—	—	1.59	5.19	1.43
frb40-19-4	40	760	0.85	—	—	—	5.83	5.18	5.26
frb40-19-5	40	760	0.85	—	—	—	4.83	—	4.37
frb45-21-1	45	945	0.86	—	—	—	105.4	—	97.81
frb45-21-2	45	945	0.86	—	—	—	62.36	8.45	62.33
frb45-21-3	45	945	0.86	—	—	—	37.42	8.39	34.49
frb45-21-4	45	945	0.86	—	—	—	30.47	—	27.56
frb45-21-5	45	945	0.86	—	—	—	189.1	—	176.9
frb50-23-1	50	1,150	0.87	—	—	—	694.7	—	640.4
frb50-23-2	50	1,150	0.87	—	—	—	306.7	—	284.8
frb50-23-4	50	1,150	0.87	—	—	—	14.76	16.08	13.92
frb50-23-5	50	1,150	0.87	—	—	—	218.1	—	205.8
frb53-24-2	53	1,272	0.88	—	—	—	176.8	—	169.9
frb53-24-3	53	1,272	0.88	—	—	—	1,845	—	1,705
frb53-24-4	53	1,272	0.88	—	—	—	—	—	3,385
frb53-24-5	53	1,272	0.88	—	—	—	897.1	—	825.3
frb56-25-2	56	1,400	0.88	—	—	—	2,359	—	2,245

Note. Boldface numbers stand for the best runtime in each line.

Table 4. Mean Search Tree Sizes (in 10^4) and Runtimes (in Seconds) of Different Versions of IncMC2 for Random Graphs with Different Numbers of Vertices (N) and Different Densities (D)

N	D	UB _{IS}		UB _{IS} + IncUB		UB _{MaxSAT}		UB _{MaxSAT} + IncUB	
		Tree	Time	Tree	Time	Tree	Time	Tree	Time
200	0.9	4,052	130.2	2,591	77.96	43.65	8.22	32.46	5.95
200	0.95	2,236	126.2	1,935	105.5	3.14	0.98	2.88	0.90
300	0.8	31,609	787.9	17,864	385.5	1,866	231.4	1,122	130.7
300	0.9	—	—	—	—	14,583	3,311	13,542	3,110
#solved			0		0		30		40
400	0.7	13,652	283.7	7,806	137.2	1,732	156.8	1,046	87.61
400	0.8	—	—	—	—	27,854	3,399	24,243	3,016
#solved			0		0		31		32
500	0.6	1,514	26.90	1,195	20.68	322.0	19.82	264.6	16.52
500	0.7	105,886	2,363	67,661	1,319	13,639	1,282	8,995	808.0
1,000	0.3	74.31	0.82	70.48	0.74	20.17	0.97	20.05	0.90
1,000	0.4	542.9	8.07	418.7	6.01	247.4	8.58	198.9	6.59
1,000	0.5	10,953	187.9	8,513	142.7	3,326	173.8	2,711	137.9
2,000	0.2	83.27	1.42	82.99	1.34	30.47	1.53	30.46	1.45
2,000	0.3	1,541	21.25	1,288	17.68	898.1	25.23	784.4	21.07
2,000	0.4	41,131	652.7	35,892	561.7	14,076	669.8	12,545	590.1
5,000	0.1	94.07	5.30	94.04	4.84	86.40	6.23	86.38	5.70
5,000	0.2	5,957	75.91	5,508	73.09	3,473	103.2	3,406	97.22
10,000	0.1	859.9	38.51	858.9	40.57	423.5	54.59	423.5	52.56
10,000	0.2	87,329	3,267	65,324	2,827	64,089	3,960	48,878	3,344
15,000	0.1	4,937	391.0	4,445	378.9	1,148	387.3	1,145	299.1
20,000	0.05	796.0	149.1	796.0	138.9	753.3	175.8	752.7	165.9
Average gain due to IncUB		Tree 25.5%		Time 31.1%		Tree 18.9%		Time 23.8%	

Note. Boldface numbers stand for the best performance in each line.

Table 5. Search Tree Sizes (in 10^4) and Runtimes (in Seconds) of Different Versions of IncMC2 to Solve DIMACS Graphs, Excluding Those Graphs That Cannot Be Solved by All Solvers Within 10^5 s

Instance	ω	UB _{IS}		UB _{IS} + IncUB		UB _{MaxSAT}		UB _{MaxSAT} + IncUB	
		Tree	Time	Tree	Time	Tree	Time	Tree	Time
brock400_1	27	23,228	558.7	13,847	305.5	2,360	228.8	1,449	131.7
brock400_2	29	21,709	525.1	11,141	230.2	2,104	207.7	1,112	97.44
brock400_3	31	28,441	746.7	8,066	135.8	2,656	305.7	835.1	73.69
brock400_4	33	12,038	376.8	2,098	35.04	1,110	142.7	237.7	20.22
brock800_1	23	463,268	12,343	127,083	2,618	78,218	7,694	23,530	1,862
brock800_2	24	553,397	14,810	127,637	2,612	92,868	9,193	22,332	1,891
brock800_3	25	253,246	7,701	53,342	1,001	43,392	4,550	10,804	737.2
brock800_4	26	278,159	7,868	94,885	1,613	46,799	4,653	17,610	1,300
C250.9	44	75,555	2,847	74,942	2,764	1,004	192.3	999.1	189.4
DSJC1000_5	15	11,376	183.0	8,197	127.7	3,487	173.0	2,683	126.5
gen200_p0.9_55	55	0.79	0.05	0.44	0.03	0.17	0.05	0.17	0.04
gen400_p0.9_55	55	4,332	305.8	2,638	168.8	0.49	1.69	0.40	1.09
gen400_p0.9_65	65	330.2	23.51	74.39	3.22	0.33	0.52	0.31	0.32
gen400_p0.9_75	75	27.19	2.27	6.37	0.48	0.34	0.29	0.34	0.21
hamming10-2	512	13.18	8.29	13.18	7.84	13.18	35.88	13.18	35.46
keller5	27	41,346	2,788	16,190	1,043	841.8	259.5	782.0	180.0
MANN_a27	126	4.99	1.24	3.53	0.69	1.22	0.28	1.20	0.25
MANN_a45	345	385.5	768.0	304.7	572.0	8.64	11.98	8.32	10.51
p_hat1000-2	46	2,128	116.0	2,126	113.5	138.8	25.85	138.8	25.69
p_hat1500-1	12	264.8	3.13	255.0	2.87	81.25	3.55	81.11	3.32
p_hat1500-2	65	94,388	8,657	93,467	8,376	2,614	868.5	2,610	865.9
p_hat300-3	36	25.32	1.03	25.16	1.00	1.41	0.33	1.41	0.32
p_hat500-2	36	4.28	0.39	4.28	0.39	0.78	0.28	0.78	0.28
p_hat500-3	50	1,447	85.87	1,439	83.58	70.44	14.38	70.43	13.82
p_hat700-2	44	44.36	2.83	42.68	2.73	3.50	1.07	3.50	1.06
p_hat700-3	62	16,157	1,306	16,157	1,334	493.9	156.1	493.9	154.6
san200_0.9_3	44	30.42	1.36	15.38	0.61	0.51	0.08	0.46	0.06
san400_0.7_3	22	6.21	0.52	4.54	0.26	0.12	0.38	0.12	0.20
san400_0.9_1	100	70.08	11.56	8.67	0.76	0.73	0.33	0.73	0.27
sanr200_0.9	42	864.9	31.42	630.3	22.23	14.36	2.56	10.74	1.84
sanr400_0.7	21	6,557	121.7	6,180	110.1	875.5	65.30	837.3	61.3
Average gain due to IncUB		Tree		Time		Tree		Time	
		129.9%		217.2%		60.5%		99.4%	

Note. Boldface numbers stand for the best performance in each line.

in Table 5. For example, UB_{IS} + IncUB is 217% faster than UB_{IS} in the average for DIMACS graphs, while this gain is only 31% for random graphs. Observe that the gain in terms of runtime is also more important than the gain in terms of search tree size for DIMACS graphs. In fact, when a branch can be pruned by IncUB in line 9 in Algorithm 4, it might also be pruned using UB_{MaxSAT} in line 5 after the recursive call. However, the pruning using IncUB is less costly than using MaxSAT reasoning.

For BHOSLIB graphs, UB_{MaxSAT} + IncUB (UB_{IS} + IncUB) roughly exhibits the same search tree as UB_{MaxSAT} (UB_{IS}), because if a branch can be pruned by IncUB, it can also be pruned by UB_{MaxSAT} (UB_{IS}) for these graphs. However, pruning by IncUB is less time-consuming, making UB_{MaxSAT} + IncUB (UB_{IS} + IncUB) faster than UB_{MaxSAT} (UB_{IS}), as shown in Table 6.

8.5. IncUB in Finding and Proving an Optimal Solution

As most BnB MaxClique algorithms, IncMC2 solves a graph G in two phases. In the first phase, IncMC2

finds an optimal solution. In the second phase, IncMC2 proves that the solution is indeed optimal by showing that no better solution exists. We denote the runtime in the first phase by *findTime*, the runtime in the second phase by *provTime*, and the whole solving time by *solvTime*. It roughly holds that $\text{solvTime} = \text{findTime} + \text{provTime}$, excluding the negligible preprocessing time.

In the usual case, IncMC2 solves a graph from scratch. In this case, we respectively denote the runtime in the first phase, the runtime in the second phase, and the whole solving time by $\text{findTime}_{\text{from0}}$, $\text{provTime}_{\text{from0}}$, and $\text{solvTime}_{\text{from0}}$. In some case, IncMC2 can also be fed with an optimal solution C_{opt} at the beginning to prove the optimality of the solution. In this case, we respectively denote the runtime in the first phase, the runtime in the second phase, and the whole solving time by $\text{findTime}_{\text{fromOpt}}$, $\text{provTime}_{\text{fromOpt}}$, and $\text{solvTime}_{\text{fromOpt}}$. It holds that $\text{findTime}_{\text{fromOpt}} = 0$ and $\text{solvTime}_{\text{fromOpt}} = \text{provTime}_{\text{fromOpt}}$.

IncMC2 solves a graph from an optimal solution C_{opt} in the same way as it solves the graph from scratch.

Table 6. Search Tree Sizes (in 10^4) and Runtimes (in Seconds) of Different Versions of IncMC2 to Solve BHOSLIB Graphs That Are Solved by at Least One Solver in 3,600 Seconds

Instance	ω	UB _{IS}		UB _{IS} + IncUB		UB _{MaxSAT}		UB _{MaxSAT} + IncUB	
		Tree	Time	Tree	Time	Tree	Time	Tree	Time
frb30-15-1	30	3.68	0.51	3.68	0.25	0.09	0.38	0.09	0.12
frb30-15-2	30	8.98	0.69	8.98	0.43	0.10	0.40	0.10	0.14
frb30-15-3	30	8.47	0.80	7.69	0.49	0.11	0.46	0.11	0.20
frb30-15-4	30	11.78	0.90	10.98	0.60	0.12	0.46	0.12	0.20
frb30-15-5	30	3.41	0.51	3.28	0.25	0.10	0.40	0.10	0.15
frb35-17-1	35	41.52	4.45	41.52	3.84	0.22	1.37	0.22	0.80
frb35-17-2	35	400.1	26.14	400.1	24.91	0.29	1.71	0.29	1.14
frb35-17-3	35	37.68	3.20	37.68	2.62	0.14	0.86	0.14	0.30
frb35-17-4	35	17.08	1.95	17.08	1.35	0.14	0.90	0.14	0.33
frb35-17-5	35	78.84	6.33	78.84	5.78	0.17	1.06	0.17	0.47
frb40-19-1	40	150.4	18.68	150.4	17.56	0.28	2.47	0.28	1.44
frb40-19-2	40	12.92	2.83	12.92	1.64	0.17	1.61	0.17	0.44
frb40-19-3	40	531.8	46.48	531.8	45.22	0.27	2.64	0.27	1.43
frb40-19-4	40	2,510	249.3	2,510	241.7	1.09	6.50	1.09	5.26
frb40-19-5	40	1,238	118.6	1,238	116.8	0.84	5.57	0.84	4.37
frb45-21-1	45	20,351	2,331	20,351	2,253	10.24	100.4	10.24	97.81
frb45-21-2	45	10,191	1,130	10,191	1,096	6.11	64.84	6.11	62.33
frb45-21-3	45	15,536	1,694	15,536	1,658	2.94	36.70	2.94	34.49
frb45-21-4	45	5,929	794.2	5,929	759.4	2.65	29.76	2.65	27.56
frb45-21-5	45	—	—	—	—	15.84	179.0	15.84	176.9
frb50-23-1	50	—	—	—	—	61.35	643.9	61.35	640.4
frb50-23-2	50	—	—	—	—	21.43	288.6	21.43	284.8
frb50-23-4	50	3,082	395.3	3,082	380.7	1.24	17.73	1.24	13.92
frb50-23-5	50	—	—	—	—	22.60	209.8	22.60	205.8
frb53-24-2	53	—	—	—	—	12.68	175.6	12.68	169.9
frb53-24-3	53	—	—	—	—	97.76	1,713	97.76	1,705
frb53-24-4	53	—	—	—	—	189.4	3,387	189.4	3,385
frb53-24-5	53	—	—	—	—	42.83	832.4	42.83	825.3
frb56-25-2	56	—	—	—	—	135.6	2,253	135.6	2,245
Average gain due to IncUB		Tree	Time	Tree	Time	Tree	Time	Tree	Time
		1.07%		29.1%		0.00%		67.8%	

Notes. Boldface numbers stand for the best performance in each line. When a solver fails to solve a graph within 3,600 s, the entry is marked with “—”.

The only difference lies in the value of $C_{\max}$ in Algorithm 4. Note that IncMC2 assigns $\text{VertexUB}[v, P, O] = |C_{\max}| - |C|$ for each vertex v removed from B_0 by the IncMaxSAT function (Algorithm 3) in line 5 and $\text{VertexUB}[v, P, O] = |C_1| - |C|$ for each branching vertex v in line 16 of Algorithm 4. When IncMC2 solves a graph from scratch and spends a lot of time to find an optimal solution, the size of $C_{\max}$ and C_1 is small at the beginning and grows slowly before reaching the optimality. In this case, VertexUB is of good quality for many vertices, making IncUB tight. However, when IncMC2 solves a graph from an optimal solution C_{opt} , $C_{\max}$ and C_1 are always C_{opt} . So, $\text{VertexUB}[v, P, O]$ is of poor quality for many vertices, making IncUB less effective for pruning branches. We conducted an experiment to confirm this intuition.

Table 7 compares $\text{findTime}_{\text{from0}}$, $\text{provTime}_{\text{from0}}$, $\text{solvTime}_{\text{from0}}$, and $\text{provTime}_{\text{fromOpt}}$ of IncMC2 when solving the hardest graphs from Table 2 (by excluding the hardest graphs: C2000.5 and $p_{\text{hat}}1000-3$) and Table 3. It also gives the search tree size of

IncMC2 when solving the graphs from scratch and from optima, as well as the number of branches IncUB prunes in line 9 of Algorithm 4 in the two cases. Three observations can be made from Table 7.

- The optimal partition of the graphs named “frbd- d ” contains d ISs. Their optimal solution is of size d . When IncMC2 is fed with an optimal solution, most vertices are inserted into A by the FilterByColoring and IncMaxSAT functions, making IncMC2 able to immediately prove the optimality.

- For graphs for which IncMC2 quickly finds an optimal solution (e.g., $\text{findTime}_{\text{from0}} < 10$ s), IncUB prunes roughly the same number of branches when it solves the graphs from scratch or from an optimal solution.

- For graphs (excluding the special graphs named “frbd- d ”) for which IncMC2 spends a long time finding an optimal solution (e.g., $\text{findTime}_{\text{from0}} > 30$ s), IncUB prunes many more branches when it solves the graphs from scratch than when it solves the graphs from an optimal solution, confirming our intuition discussed

Table 7. Solving Time (in Seconds), Search Tree Size (in 10^4), and Number of Branches (in 10^4) IncUB Cuts When IncMC2 Solves a Graph from Scratch and from Optima

Instance	Solving from scratch					Solving from optima		
	findTime _{from0}	provTime _{from0}	solvTime _{from0}	Tree size	#IncUBcut	provTime _{fromOpt}	Tree size	#IncUBcut
brock400_1	129.6	2.14	131.7	1,449	43.56	121.1	1,111	0.02
brock400_2	97.43	0.01	97.44	1,112	40.14	56.03	454.8	0.00
brock400_3	73.37	0.32	73.69	835.1	98.42	59.08	356.1	0.09
brock400_4	19.80	0.42	20.22	237.7	49.48	30.05	161.6	0.03
brock800_1	826.0	1,036	1,862	23,530	5,461	6,162	58,331	39.19
brock800_2	1,839	52.91	1,891	22,332	3,374	5,154	45,984	30.19
brock800_3	603.5	133.7	737.2	10,804	3,333	2,927	24,477	13.89
brock800_4	1,229	71.04	1,300	17,610	2,693	2,147	16,773	6.76
C250.9	0.30	189.1	189.4	999.1	0.76	194.4	1,003	0.04
DSJC1000_5	35.26	91.27	126.5	2,683	21.49	163.2	3,167	1.89
gen400_p0.9_55	1.09	0.00	1.09	0.40	0.05	0.04	0.04	0.04
gen400_p0.9_65	0.32	0.00	0.32	0.31	0.06	0.04	0.04	0.04
gen400_p0.9_75	0.21	0.00	0.21	0.34	0.04	0.05	0.04	0.04
MANN_a45	5.14	5.37	10.51	8.32	0.10	7.40	2.46	0.10
keller5	34.54	145.4	180.0	782.0	147.2	205.5	727.2	12.88
p_hat1000-2	0.17	25.52	25.69	138.8	0.08	26.07	138.7	0.00
p_hat1500-2	0.82	865.1	865.9	2,610	2.46	890.0	2,613	0.01
p_hat700-3	1.46	153.1	154.6	493.9	0.01	158.8	493.8	0.01
sanr400_0.7	1.29	60.06	61.35	837.3	13.03	64.30	866.7	0.11
frb50-23-1	640.4	0.00	640.4	61.35	0.12	0.38	0.12	0.12
frb50-23-2	284.8	0.00	284.8	21.43	0.11	0.37	0.12	0.12
frb50-23-4	13.92	0.00	13.92	1.24	0.12	0.38	0.12	0.12
frb50-23-5	205.8	0.00	205.8	22.60	0.15	0.38	0.12	0.12
frb53-24-2	169.9	0.00	169.9	12.68	0.13	0.47	0.13	0.13
frb53-24-3	1,705	0.00	1,705	97.76	0.13	0.48	0.13	0.13
frb53-24-4	3,385	0.00	3,385	189.4	0.18	0.48	0.13	0.13
frb53-24-5	825.3	0.00	825.3	42.83	0.29	0.48	0.13	0.13
frb56-25-2	2,245	0.00	2,245	135.6	1.04	0.58	0.14	0.14

Note. Boldface numbers stand for the best performance in each line.

above. In fact, Table 5 shows that IncUB significantly speeds up IncMC2 for these graphs. In particular, IncUB is so effective for solving brock400_4, brock800_1, brock800_2, brock800_3, brock800_4, DSJC1000_5, and keller5 from scratch, that solvTime_{from0} is even shorter than provTime_{fromOpt} for these graphs.

9. Conclusion

State-of-the-art BnB MaxClique algorithms usually do not perform searches incrementally, in the sense that, apart from the largest clique found so far, they usually do not exploit any result of previous search. In this paper, we propose to record intermediate search results for each vertex in an array called VertexUB, and then define an incremental upper bound called IncUB for a BnB MaxClique algorithm. IncUB is based on a static vertex ordering and is derived in linear time from the previous results recorded in VertexUB. So, the tightness of IncUB depends on the quality of VertexUB.

Then, we present three methods to compute VertexUB for a vertex: by independent set partition, by incremental MaxSAT reasoning, and by making a BnB algorithm branch on the vertex. We combine the three methods in a BnB MaxClique algorithm called IncMC2. Experimental results on standard benchmarks show that IncUB substantially speeds up IncMC2 and makes

IncMC2 faster than other BnB algorithms such as MCS and BBMC. In particular, it can even make IncMC2 faster for solving some graphs from scratch than for proving the optimality of an optimal solution of these graphs.

Acknowledgments

The authors thank the anonymous reviewers for their constructive comments and suggestions that helped to improve this paper.

Endnotes

¹ Available at <http://cs.hbg.psu.edu/txn131/clique.html>.

² Available at <http://www.nlsde.buaa.edu.cn/~kexu/benchmarks/graph-benchmarks.htm>.

³ Available at <http://www.sicmm.org/~konc/maxclique/>.

References

- Benlic U, Hao JK (2013) Breakout local search for maximum clique problems. *Comput. Oper. Res.* 40(1):192–206.
- Berman P, Pelc A (1990) Distributed probabilistic fault diagnosis for multiprocessor systems. *Proc. 20th Annual Internat. Sympos. Fault-Tolerant Comput., FTCS '90* (IEEE Computer Society, Washington, DC), 340–346.
- Cai S, Su K, Sattar A (2011) Local search with edge weighting and configuration checking heuristics for minimum vertex cover. *Artificial Intelligence* 175(9):1672–1696.
- Carraghan R, Pardalos PM (1990) An exact algorithm for the maximum clique problem. *Oper. Res. Lett.* 9(6):375–382.

- Fahle T (2002) Simple and fast: Improving a branch-and-bound algorithm for maximum clique. Di Battista G, Zwick U, eds. *Proc. 11th Annual Eur. Sympos. Algorithms, ESA '03* (Springer, Berlin), 485–498.
- Grosso A, Locatelli M, Pullan W (2008) Simple ingredients leading to very efficient heuristics for the maximum clique problem. *J. Heuristics* 14(6):587–612.
- Konc J, Janezic D (2007) An improved branch and bound algorithm for the maximum clique problem. *Comm. Math. Comput. Chemistry* 58:569–590.
- Li CM, Quan Z (2010a) Combining graph structure exploitation and propositional reasoning for the maximum clique problem. *Proc. 22nd IEEE Internat. Conf. Tools with Artificial Intelligence, ICTAI '10*, Vol. 1 (IEEE Computer Society, Washington, DC), 344–351.
- Li CM, Quan Z (2010b) An efficient branch-and-bound algorithm based on maxsat for the maximum clique problem. *Proc. 24th AAAI Conf. Artificial Intelligence, AAAI '10* (AAAI Press, Menlo Park, CA), 128–133.
- Li CM, Fang Z, Xu K (2013) Combining MaxSAT reasoning and incremental upper bound for the maximum clique problem. *Proc. 25th IEEE Internat. Conf. Tools with Artificial Intelligence, ICTAI '13* (IEEE Computer Society, Washington, DC), 939–946.
- Li CM, Jiang H, Xu R-C (2015) Incremental MaxSAT reasoning to reduce branches in a branch-and-bound algorithm for MaxClique. Dhaenens C, Jourdan L, Marmion M-E, eds. *Proc. 9th Internat. Conf. Learn. Intelligent Optim., LION 9* (Springer International, Cham, Switzerland), 268–274.
- Li CM, Zhu Z, Manyà F, Simon L (2011) Minimum satisfiability and its applications. Walsh T, ed. *Proc. 22nd Internat. Joint Conf. Artificial Intelligence, IJCAI '11* (AAAI/IJCAI, Menlo Park, CA), 605–610.
- Li CM, Zhu Z, Manyà F, Simon L (2012) Optimizing with minimum satisfiability. *Artificial Intelligence* 190:32–44.
- McCreesh C, Prosser P (2014) Reducing the branching in a branch and bound algorithm for the maximum clique problem. O'Sullivan B, ed. *Proc. Internat. Conf. Principles Practice Constraint Programming, CP '14* (Springer International, Cham, Switzerland), 549–563.
- McCreesh C, Prosser P (2015) The shape of the search tree for the maximum clique problem and the implications for parallel branch and bound. *ACM Trans. Parallel Comput.* 2(1):Article no. 8.
- Östergård P (2002) A fast algorithm for the maximum clique problem. *Discrete Appl. Math.* 120(1–3):197–207.
- Prosser P (2012) Exact algorithms for maximum clique: A computational study. *Algorithms* 5(4):545–587.
- Pullan W, Hoos HH (2006) Dynamic local search for the maximum clique problem. *J. Artificial Intelligence Res.* 25:159–185.
- Pullan W, Mascia F, Brunato M (2011) Cooperating local search for the maximum clique problem. *J. Heuristics* 17(2):181–199.
- Régin JC (2003) Using constraint programming to solve the maximum clique problem. Rossi F, ed. *Proc. Internat. Conf. Principles Practice Constraint Programming, CP '03* (Springer, Berlin), 634–648.
- San Segundo P, Rodríguez-Losada D, Jiménez A (2011) An exact bit-parallel algorithm for the maximum clique problem. *Comput. Oper. Res.* 38(2):571–581.
- San Segundo P, Lopez A, Batsyn M, Nikolaev A, Pardalos PM (2016) Improved initial vertex ordering for exact maximum clique search. *Appl. Intelligence* 45(3):868–880.
- Strickland DM, Barnes E, Sokol JS (2005) Optimal protein structure alignment using maximum cliques. *Oper. Res.* 53(3):389–402.
- Tomita E, Kameda T (2007) An efficient branch-and-bound algorithm for finding a maximum clique with computational experiments. *J. Global Optim.* 37(1):95–111.
- Tomita E, Seki T (2003) An efficient branch-and-bound algorithm for finding a maximum clique. *Discrete Math. Theoret. Comput. Sci. Lecture Notes in Computer Science*, Vol. 2731 (Springer, Berlin), 278–289.
- Tomita E, Sutani Y, Higashi T, Takahashi S, Wakatsuki M (2010) A simple and faster branch-and-bound algorithm for finding a maximum clique. Rahman MdS, Fujita S, eds. *Proc. 4th Internat. Workshop Algorithms Comput., WALCOM '10* (Springer, Berlin) 191–203.
- Wang K (2013) Personal communication. Email from Kai Wang to the first author on March 8, 2013.
- Wu Q, Hao JK (2015) A review on algorithms for maximum clique problems. *Eur. J. Oper. Res.* 242(3):693–709.
- Xu K, Li W (2000) Exact phase transitions in random constraint satisfaction problems. *J. Artificial Intelligence Res.* 12:93–103.
- Xu K, Boussemart F, Hemery F, Lecoutre C (2007) Random constraint satisfaction: Easy generation of hard (satisfiable) instances. *Artificial Intelligence* 171(8–9):514–534.